

Equiper — plan to reach 9/10

Version 1.0 · 16 September 2026 · owner: engineering

This plan turns the findings of the full-system review into ordered work. Every item states the measured **current state**, the **target**, and the **action**. Scores are out of 10. Effort figures are rough single-developer estimates.

The measurements behind every claim are listed in the appendix. They come from running the test suite, requesting all 99 GET routes as each role, reading the settings and sync package, and `manage.py check / makemigrations --check`.

1. Where the software stands

Area	Now	Target	What closes the gap
Feature scope	8.5	9	Nothing structural; finish the broken pages below
UI consistency	8	9	Retire page-specific CSS into the design system
Correctness	4	9	Fix 18 failing routes; add a route smoke test
Security	5	9	Rotate committed keys; escape output; central authorization
Caching	3	9	The cache layer is configured but unused; make it real and fail-soft
Data model	7	9	Add the missing indexes; keep migrations in lockstep
Testing / CI	4	9	CI pipeline; tests for approval, calibration, PPM, updates
Architecture	4.5	8	Split the giant functions; one build script; finish sync refactor
Frontend weight	5.5	9	One copy of each vendor library; hashed static files
Offline / ops	7	9	Fail-soft Redis; HQ off free tier; monitoring
Documentation	6	8	A real README and a runbook
Overall	6	9	Phases 0-3 below

2. What 9/10 means here

A concrete definition, so the score is testable rather than a feeling.

Area	9/10 criteria
Correctness	Every route returns under 500 for every role, enforced by a test that runs in CI
Security	No secret in the repo or its history; output escaped by default; one authorization path; brute-force protection on login
Caching	Dashboard and report aggregates cached with explicit invalidation; the app keeps working when Redis is down; static files served with hashed names and long max-age
Testing	CI green on every push; the four business-critical flows covered end to end; sync tests run in CI

Area	9/10 criteria
Architecture	No function over 150 lines in application code; one build script; sync engine composed of single-responsibility objects
Frontend	One copy of each vendor library; no page ships CSS that duplicates the design system
Ops	Errors reach a dashboard within a minute; sync backlog alerts; HQ has no sleep window

3. Phase 0 — stop the bleeding (week 1, est. 4-5 days)

These are user-visible failures and exposed credentials. Nothing else should start before this phase lands.

3.1 Eighteen routes return HTTP 500 (est. 1.5 days)

Measured by requesting all 99 GET routes while logged in. Causes:

- **Four missing templates.** Calibration reports, calibration analytics, parameter create and inventory summary render template names that do not exist. The parameter view is worse: it renders `Calibration/parameter_form.html` in one place and `Calibration/parameter_form.html` in another, and neither file exists. Decide one spelling, create the template, and delete the other path.
- `/calibration/sessions/` calls the default Django user manager, but the project swapped in `accounts.CustomUser`. Use `get_user_model()`.
- `/Inventory/add_inventory/` returns `None` instead of a response on at least one branch.
- `/dashboard/nurse_ppms/` assumes the user has a department (`'NoneType'` object has no attribute `'name'`). Guard it, and decide what a department-less nurse should see.
- `/calibration/backup/` reverses a URL name (`calsoft_dashboard`) that does not exist, so the backup entry point is unreachable. This one matters twice over: it is the user's route to a backup.
- `Inventory/inventory_dashboard.html` uses a `plotly_app` tag that is never loaded, so the template cannot compile.

3.2 A route smoke test (est. 0.5 day)

One test that walks every named URL for each role and asserts the response is under 500. This would have caught all six causes above. Make it a required CI check so this class of bug cannot return.

3.3 Rotate the committed credentials (est. 0.5 day, plus coordination)

`config.py` ships a real sync auth token and HQ API key in its defaults, and they have been in git since the initial commit. `SECURITY.md` already lists them as needing rotation; that has not happened. Actions:

1. Rotate both on the HQ side; issue per-client sync tokens.
2. Remove the literals from `config.py` — read from environment or `~/cirqen/data/config.json` only, and fail loudly when absent.
3. Treat the old values as burned. Purging git history is optional but the rotation is not.

3.4 Debug off in shipped builds (est. 0.5 day)

`config.py` defaults `app.debug` to `True`, so a packaged client runs with stack traces and settings exposed unless `DJANGO_DEBUG=0` is set. Flip the default to `False`, set `DJANGO_DEBUG=0` in the installer, and add a build check that fails the packaging step when `DEBUG` resolves true.

3.5 Duplicate URL mounts (est. 0.5 day)

`parts_tools` is mounted at both `/accessories/` and `/parts-tools/`, and `machineReports` at both `/machineReports/` and `/`. Django warns (`urls.W005`) that reversing may break. Pick one mount each and redirect the old path.

4. Security — 5/10 to 9/10 (phase 1, est. 8-10 days)

4.1 Escape what reaches the page (est. 2 days)

- **11 uses of** `|safe`, all injecting JSON into inline `<script>` blocks on the dashboards and report hub. If any string field ever contains `</script>`, that is script injection. Replace with `{{ data|json_script:"id" }}` and read it with `JSON.parse(document.getElementById(...).textContent)`.
- **256 innerHTML assignments** in the app's own JavaScript, with escaping helpers in only 9 files. Equipment names, notes and remarks are free text typed by technicians, so this is a stored-XSS path. Route every interpolation through one shared `escapeHTML`, or switch those builders to `textContent/createElement`.

4.2 One authorization path (est. 3 days)

There are **214 inline** `role == "HOD"` **style comparisons** and only one module defining the `role_required / hod_required` decorators. Role gating does work today (a technician correctly gets 403 on user management), but every new view re-implements it.

1. Replace inline comparisons with the decorator.
2. Add one queryset-scoping helper (`for_user(qs, user)`) that filters by workshop and department, and use it in every list and detail view.
3. Add tests that a technician cannot reach another workshop's records by UUID.

4.3 Brute-force protection on login (est. 1 day)

There is **no lockout or rate limit on the login form** — the `attempts / max_attempts` fields belong to the password-reset code model, not to login. Add attempt throttling (django-axes, or a small cache-backed counter once 5.2 below lands), and rate-limit the reset-code verify endpoint, which currently accepts a 6-digit code with three attempts per code but no per-IP limit.

4.4 Remove the CSRF exemptions (est. 0.5 day)

Ten `csrf_exempt` decorators, including user create, update and delete. They are layered behind `login_required`, `role_required` and method limits, so this is not wide open — but the exemption buys nothing for same-origin calls. Send the CSRF token from the fetch calls and drop the decorator.

4.5 Error monitoring (est. 1 day)

No Sentry, no OpenTelemetry. Failures only reach rotating log files, which is exactly why 18 broken pages went unnoticed. Add Sentry (self-hosted or SaaS) for the Django process and the sync agent, with release tagging tied to the update version. Then replace the **1,335** `print()` **calls** in application code with logger calls — several currently dump user data to stdout.

4.6 Raw SQL discipline (est. 1 day)

55 `execute()` calls use f-strings, all in `sync/` (10 files), `helper_scripts/` (8), `bulider_tools/` (2) and `core/` (1) — none in the Django views. What is interpolated is table and schema identifiers from the configured sync table list, with values passed as parameters, so this is not user-facing injection today. Harden it anyway: assert every identifier against the known table allowlist before formatting, and use `psycopg2.sql.Identifier`.

4.7 Transport and headers (est. 0.5 day)

SESSION_COOKIE_SECURE and CSRF_COOKIE_SECURE are False, with no SECURE_SSL_REDIRECT or HSTS. Acceptable for a LAN-only desktop deployment, wrong the moment anything is exposed. Make them environment-driven so a hosted deployment turns them on without a code change.

Already good, keep it: secret key from the environment, Django's password validators, HttpOnly and SameSite cookies, one-hour sessions, update_session_auth_hash on password change, clickjacking and nosniff headers, Excel upload extension and size checks, and a real audit log.

5. Caching — 3/10 to 9/10 (phase 1, est. 5-6 days)

The cache layer is **configured but unused**: three caches are defined (Redis db0 for general use with zlib compression and a JSON serializer, Redis db1 for sessions, and a 24-hour file-based offline cache), yet the codebase contains

zero cache.get / cache.set **calls**, **zero template fragment caching** and **zero** cache_page. The only cache decorators in use are three never_cache calls on auth views. CACHE_MIDDLEWARE_SECONDS and CACHE_MIDDLEWARE_KEY_PREFIX are set while no cache middleware is installed, so those two settings do nothing.

5.1 Make Redis failure non-fatal (est. 1 day) — do this first

SESSION_ENGINE is cached_db against the sessions cache, and neither cache sets IGNORE_EXCEPTIONS or a socket timeout. If the bundled Redis on port 7788 is slow to start or dies, session reads raise or hang and the app becomes unusable — on a single-machine offline deployment that is a self-inflicted outage. Actions:

- Set IGNORE_EXCEPTIONS: True and SOCKET_CONNECT_TIMEOUT / SOCKET_TIMEOUT (0.2-0.5s) on both Redis caches, so a stalled Redis degrades to a miss.
- Add DJANGO_REDIS_LOG_IGNORED_EXCEPTIONS = True so degradation is visible.
- Add a startup health check that logs plainly when the cache is unavailable.
- Decide the session story: keep cached_db (durable, cache is only a read accelerator) and verify a Redis-down start still authenticates.

5.2 Cache what is expensive (est. 2 days)

Cache the aggregates, not pages — the dashboards are the slow part and their inputs change on a schedule, not per request.

What	Key	TTL	Invalidate on
Workshop dashboard counters	dash:{workshop}:{date}	5 min	Equipment, job card or PPM save
HOD system overview	dash:hod:{date}	5 min	Same, any workshop
PPM status distribution	ppm:{workshop}:{month}	10 min	PPM schedule save
Calibration schedule snapshot	cal:{workshop}:{month}	10 min	Schedule or session save
Report hub technician table, per workshop	rpt:{type}:{period}	15 min	Job card status change
Equipment category chart	inv:cat:{workshop}	30 min	Equipment save

Implement with `cache.get_or_set` behind one small helper so the key prefix and TTL policy live in one place, and clear the relevant keys from `post_save` / `post_delete` signals rather than waiting for the TTL.

5.3 Use the offline cache for what it was made for (est. 1 day)

The file-based offline cache is defined and never used. Point it at the data the UI needs when HQ is unreachable — last known sync status, HQ version, update availability — so those panels render instantly and do not block on a network call.

5.4 Hashed static files (est. 1 day)

There is no `ManifestStaticFilesStorage` and no `WhiteNoise`, so CSS and JS are served under stable names. Because the app ships in-app updates that replace those files, a browser holding a cached copy shows a stale UI after an update — the current answer is "tell the user to hard-refresh". Actions:

- Use Django 5's `STORAGES` with `ManifestStaticFilesStorage`, run `collectstatic` in the packaging step, and serve `/static/` with a long `max-age` plus `immutable`.
- Keep `index.html`-style responses uncached (`never_cache` where relevant).
- Verify the update flow: new build, new hashes, no hard refresh needed.

5.5 Query budget (est. 1 day)

195 uses of `select_related` / `prefetch_related` show the habit exists, but there are still **40 unbounded** `objects.all()` **calls in view modules** and 68 `Paginator` uses. Add a debug-only assertion (or `django-silk` in development) that fails a request over a query threshold, then fix the worst offenders — the inventory and job card lists first.

6. Testing and CI — 4/10 to 9/10 (phase 1-2, est. 6-8 days)

197 tests pass in 3.2 seconds, which is a real asset. The gaps:

- **No CI at all** — no workflow, no pre-commit, no linter config. Add GitHub Actions running `manage.py test`, the sync pytest suite, and `ruff`. Make the route smoke test (3.2) a required check. (est. 1 day)
- **The 49 sync tests never run in the default suite.** `sync` is not an installed app and the tests are pytest-only, so `manage.py test` collects none of them; they also skip silently when the embedded Postgres binaries are absent. Add a `pytest.ini`, run them in CI with a Postgres service, and make the skip loud. (est. 1 day)
- **Zero tests** for job cards, dashboard, report hub, machine reports, parts & tools, updates and the audit log. Cover the four flows that carry compliance and money first: job card approval, calibration session to certificate, PPM scheduling, and update apply/rollback. (est. 4-5 days)

7. Architecture — 4.5/10 to 8/10 (phase 2-3, est. 10-12 days)

- **192 functions over 100 lines.** Worst in application code: `apply_remote_update_locally` (684), job card technician approval (463), nurse approval (328). Split along the seams the names already suggest: `validate`, `persist`, `notify`. (est. 4 days)
- **Three build scripts** — `build_backup.py` (5,484 lines), `bulid_v1.py` (4,348) and `builder/bulid_backup.py` (3,972) — with different checksums, so they have drifted. Keep one, delete the others, and fix the `bulid` typo. (est. 2 days)
- **Sync engine across nine mixin files** plus capability mixins. The architecture doc is good and `CODE_REVIEW.md` already found that the audited outbox engine never runs in production while a third

implementation does. Continue the refactor that review recommends: Uploader, Downloader, Applier as objects. (est. 4 days, do it behind the sync tests)

- **686 TODO/FIXME markers and 71 bare except: clauses.** Triage the TODOs into issues or delete them; make every bare except name its exception. (est. 1-2 days)

8. Frontend — 8/10 to 9/10 (phase 2, est. 4-5 days)

The design system landed this week: shared tokens, one card, one table, one tab style, dark mode, and pages that fill the screen. What remains:

- **Duplicate vendor libraries:** two copies of Chart.js (532K combined) and four of select2 (~470K), plus jQuery, Bootstrap bundle, DataTables, flatpickr, Font Awesome and boxicons. Keep one of each; drop DataTables and flatpickr if the shared components can cover their pages. (est. 2 days)
- **95 page stylesheets, 68k lines.** Fold the page-specific rules that now duplicate the design system, and delete the five unused stylesheets and the unreferenced calsoft.js. (est. 2 days)
- **49 of 75 templates carry inline <script>, and there are 120 inline onclick handlers.** Move them to delegated listeners in module files — this also removes the main obstacle to a strict Content-Security-Policy. (est. 1 day, pairs with 4.1)
- **Accessibility:** the baseline is decent (137 aria-labels, 253 labels bound to inputs, alt text on all images). Do one keyboard and contrast pass over the custom dropdowns and the calibration entry tables.
- **i18n decision:** USE_I18N is on with zero translation tags. Either wrap strings or turn the flag off; leaving it half-done misleads.

9. Offline and operations — 7/10 to 9/10 (phase 2-3, est. 4-5 days)

- **HQ runs on a free tier that sleeps.** Your own PHASE4_OPS.md names this the real single point of failure and notes the client-side retry only softens it. Move the service to a paid instance. (est. 0.5 day, plus cost)
- **No alerting on sync health.** The agent writes a status file and logs; add an alert when the outbox backlog or the time since last successful upload crosses a threshold. (est. 1.5 days)
- **The updates app has no tests** yet applies migrations on client machines. Add a test that applies and rolls back a package against a throwaway database. (est. 2 days, counted in section 6)
- **Backups:** fix the broken page (3.1), then verify a restore on a scratch machine and write the steps down. (est. 1 day)

10. Documentation — 6/10 to 8/10 (est. 2 days)

The module documentation is genuinely good: the calibration technical document, the sync architecture and phase notes, SECURITY.md and CODE_REVIEW.md. The README is one line. Write a README that covers what the system is, how to run it for development, how to run the tests, and how a release is built; add a runbook for the three things that go wrong in the field (Redis down, HQ unreachable, a failed update).

11. Timeline

Phase	Focus	Est. effort
0	Broken routes, smoke test, credential rotation, debug default, URL mounts	4-5 days
1	Security hardening, caching, CI and the sync test wiring	3-4 weeks
2	Business-flow tests, frontend diet, build-script collapse, ops alerting	3-4 weeks
3	Sync refactor, function splitting, documentation, accessibility pass	2-3 weeks

Roughly eight to eleven weeks for one developer, and phase 0 is the part that should not wait.

12. Acceptance checklist for 9/10

- [] Every route returns under 500 for every role, checked in CI
- [] No secret in the working tree; HQ keys rotated and per-client
- [] Packaged builds fail to ship if `DEBUG` resolves true
- [] No `|safe` on untrusted data; every `innerHTML` interpolation escaped
- [] One authorization decorator and one queryset scoper, with cross-role tests
- [] Login attempts throttled; reset-code verification rate-limited
- [] Dashboard and report aggregates cached with explicit invalidation
- [] The app starts and authenticates with Redis stopped
- [] Static files served under hashed names; no hard refresh after an update
- [] CI green on push: Django tests, sync tests, linter, smoke test
- [] The four business-critical flows covered end to end
- [] Errors visible in a dashboard within a minute of happening
- [] One build script; no application function over 150 lines
- [] One copy of each vendor library
- [] README and field runbook in place

Appendix — measurements behind this plan

Taken on 16 September 2026 against the working tree.

Measurement	Value
Python / templates / CSS / JS	425 files 116k lines / 75 files 17k / 95 files 68k / 193 files 196k
Models, relations, indexes	62 models, 103 FK/O2O, 117 indexed fields, 51 UUID PKs
Migration state	<code>makemigrations --check</code> : no changes pending
Tests	197 pass in 3.2s; 256 <code>def test_</code> exist; sync's 49 not collected
Routes	99 GET routes; 18 return 500 for a logged-in user

Measurement	Value
System check	1 warning: duplicate <code>partstools</code> URL namespace
Cache	3 caches configured; 0 <code>cache.get/cache.set</code> ; 0 fragment caches
Authorization	214 inline role comparisons; 2 files using decorators
Output escaping	11 uses of the <code>safe</code> filter; 256 <code>innerHTML</code> assignments; 9 files with escaping helpers
Raw SQL	55 <code>f-string execute()</code> calls, none in Django views
Logging vs printing	4,131 logger calls, 1,335 <code>print()</code> calls
Complexity	192 functions over 100 lines; largest 1,760
Markers	686 TODO/FIXME; 71 bare <code>except :</code>
Static assets	7.7MB; duplicate <code>Chart.js</code> and <code>select2</code> copies
Tracked bloat	34MB <code>inv_asset</code> including vendored Dash/React source maps